

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Львівська політехніка»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра «Системи штучного інтелекту»



ЛАБОРАТОРНА РОБОТА №2

з дисципліни

«Штучний інтелект та інноваційні інформаційні технології»

на тему

Побудова та аналіз моделей штучного інтелекту з використанням PyTorch

Виконав:

студент групи КНСШ-13

Галич Максим

Викладач:

доц. каф. СШІ к.т.н.,

Думин І.Б.

Львів — 2026

Мета роботи. Опанувати практику створення й оптимізації моделей машинного навчання у PyTorch, навчитися застосовувати різні архітектури, регуляризацію та методи покращення узагальнюючої здатності моделі, виконати порівняльний аналіз результатів, зокрема порівняти PyTorch із TensorFlow.

1. Постановка експерименту

Лабораторна робота №2 відтворює засобами PyTorch той самий експеримент, що був виконаний у лабораторній роботі №1 засобами TensorFlow. Використано той самий набір даних **California Housing** (20 640 записів, 8 ознак, задача регресії), той самий розвідувальний аналіз і ту саму попередню обробку.

Щоб порівняння фреймворків було коректним, усі параметри експерименту винесено у спільний модуль `src/config.py`, який імпортують обидві реалізації. Таким чином, будь-яка різниця в результатах зумовлена фреймворком, а не розбіжністю в налаштуваннях.

```
1  """Shared hyperparameters for both labs.
2
3  Both frameworks import these exact values, so any difference between the
4  TensorFlow and the PyTorch results comes from the framework itself and not
5  from a mismatch in the experiment setup.
6  """
7
8  from pathlib import Path
9
10 SEED = 42
11
12 # Sample split: 60 / 20 / 20
13 TEST_SIZE = 0.20
14 VAL_SIZE = 0.25 # 0.25 of the remaining 80% yields 20% of the full dataset
15
16 # Training (task 8: 10-20 epochs, fixed optimizer and learning rate)
17 EPOCHS = 20
18 BATCH_SIZE = 256
19 LEARNING_RATE = 1e-3
20 L2_LAMBDA = 1e-4
21 DROPOUT_RATE = 0.2
22
23 # Architectures (task 6)
24 HIDDEN_UNITS = (64, 32)
25
26 # Activations to compare (task 7)
27 ACTIVATIONS = ("relu", "gelu")
```

```

28
29 # Number of quantile groups of the target used for the imbalance analysis
30 TARGET_BINS = 5
31
32 # src/aiit/config.py -> src/aiit -> src -> the subject directory
33 ROOT = Path(__file__).resolve().parent.parent.parent
34 REPO_ROOT = ROOT.parent.parent.parent # the workspace root
35
36 # Where this subject sits under src/, e.g. "1th term 2026 Autumn/<subject>".
37 # Both the artifact tree and the log tree mirror it, so the term and subject
38 # names follow the actual directory layout instead of being duplicated here.
39 SUBJECT_PATH = ROOT.relative_to(REPO_ROOT / "src")
40
41 # Every generated artifact (figures, metrics) lands in .cache/ – the workspace's
42 # shared, untracked tree of intermediate data. The source tree stays clean.
43 ARTIFACTS = REPO_ROOT / ".cache" / "reports-artifacts" / SUBJECT_PATH
44
45 # Per-run log files, one directory per lab, in the workspace's shared logs/ tree.
46 LOGS = REPO_ROOT / "logs" / SUBJECT_PATH
47
48 EDA = ARTIFACTS / "eda"
49 LAB1 = ARTIFACTS / "lab1"
50 LAB2 = ARTIFACTS / "lab2"

```

Лістинг 1: Спільні гіперпараметри обох лабораторних робіт (src/config.py)

Табл. 1: Умови експерименту, однакові для обох фреймворків

Параметр	Значення
Поділ вибірки	60/20/20 зі стратифікацією
Оптимізатор	Adam
Швидкість навчання	10^{-3}
Розмір батча	256
Кількість епох	20
Функція втрат	MSE
Зерно генератора	42
Обчислювальний пристрій	CPU

2. Реалізація архітектур у PyTorch

Реалізовано ті самі дві архітектури: базову feedforward мережу з двома прихованими шарами та ускладнену мережу з Dropout, Batch Normalization і L2-регуляризацією. Кожну архітектуру навчено з активаціями ReLU та GELU. Принципова відмінність від Keras полягає у способі задання L2-регуляризації. У Keras вона вказується на рівні шару через `kernel_regularizer`, натомість у PyTorch її задає оптимізатор параметром `weight_decay`.

```
26 def build_baseline(input_dim: int, activation: str) -> nn.Module:
27     """Baseline feedforward network: two hidden layers, no regularization."""
28     return nn.Sequential(
29         nn.Linear(input_dim, HIDDEN_UNITS[0]),
30         _activation(activation),
31         nn.Linear(HIDDEN_UNITS[0], HIDDEN_UNITS[1]),
32         _activation(activation),
33         nn.Linear(HIDDEN_UNITS[1], 1),
34     )
35
36
37 def build_regularized(input_dim: int, activation: str) -> nn.Module:
38     """Extended network: Dropout + BatchNorm1d + L2 (through weight_decay)."""
39     layers: list[nn.Module] = []
40     prev = input_dim
41
42     for units in HIDDEN_UNITS:
43         layers += [
44             nn.Linear(prev, units, bias=False),
45             nn.BatchNorm1d(units),
46             _activation(activation),
47             nn.Dropout(DROPOUT_RATE),
48         ]
49         prev = units
50
51     layers.append(nn.Linear(prev, 1))
52     return nn.Sequential(*layers)
```

Лістинг 2: Архітектури моделей у PyTorch (`src/torch_models.py`)

3. Цикл навчання

Друга принципова відмінність — відсутність у PyTorch готового методу `.fit()`. Цикл навчання доводиться писати вручну: перебір епох і батчів, обнулення градієнтів, прямий прохід, обчислення втрати, зворотне поширення та крок оптимізатора.

Окремої уваги потребує перемикання режимів `model.train()` та `model.eval()`. Без нього Dropout лишався б активним під час обчислення валідаційної втрати, а BatchNorm використовував би статистики поточного батча замість накопичених, що спотворило б результати.

```

57 def train_one(model: nn.Module, weight_decay: float, ds: dict) -> tuple[dict, float]:
58     """A manual training loop – unlike Keras, PyTorch has no .fit()."""
59     criterion = nn.MSELoss()
60     optimizer = torch.optim.Adam(
61         model.parameters(), lr=LEARNING_RATE, weight_decay=weight_decay
62     )
63
64     train_loader = make_loader(ds["X_train"], ds["y_train"], shuffle=True)
65     X_val = torch.from_numpy(ds["X_val"])
66     y_val = torch.from_numpy(ds["y_val"]).unsqueeze(1)
67
68     history: dict[str, list[float]] = {"loss": [], "val_loss": []}
69     start = time.perf_counter()
70
71     for _ in range(EPOCHS):
72         model.train()
73         running, seen = 0.0, 0
74         for xb, yb in train_loader:
75             optimizer.zero_grad()
76             loss = criterion(model(xb), yb)
77             loss.backward()
78             optimizer.step()
79             running += loss.item() * len(xb)
80             seen += len(xb)
81
82         model.eval()
83         with torch.no_grad():
84             val_loss = criterion(model(X_val), y_val).item()
85
86         history["loss"].append(running / seen)
87         history["val_loss"].append(val_loss)
88
89     return history, time.perf_counter() - start

```

Лістинг 3: Ручний цикл навчання у PyTorch (src/run_lab2.py)

4. Результати PyTorch

4.1. Метрики якості

Табл. 2: Метрики моделей на тестовій вибірці (PyTorch)

Модель	RMSE	MAE	R^2	MAPE, %	Час, с
baseline-relu	0,6165	0,4313	0,7114	24,0	4,20
baseline-gelu	0,6228	0,4322	0,7055	23,8	6,12
regularized-gelu	0,7495	0,4948	0,5734	23,2	15,72
regularized-relu	0,7542	0,4952	0,5681	22,9	4,56

Як і у TensorFlow, найкращою виявилася базова мережа з активацією ReLU: $RMSE = 0,617$, $R^2 = 0,711$.

4.2. Навчальні криві

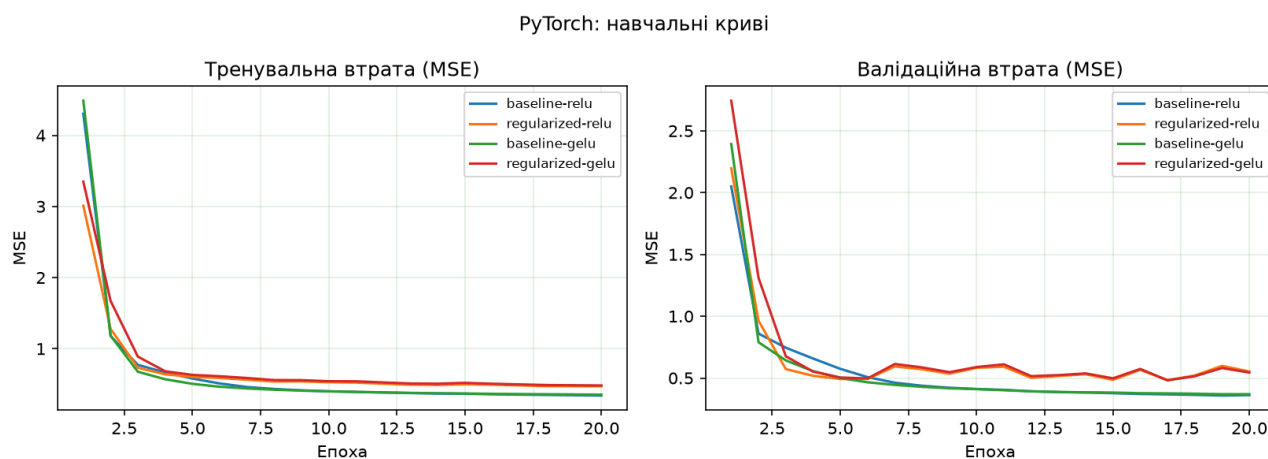


Рис. 1: PyTorch: криві тренувальної та валідаційної втрати



Рис. 2: *PyTorch: розрив між валідаційною та тренувальною втратою*

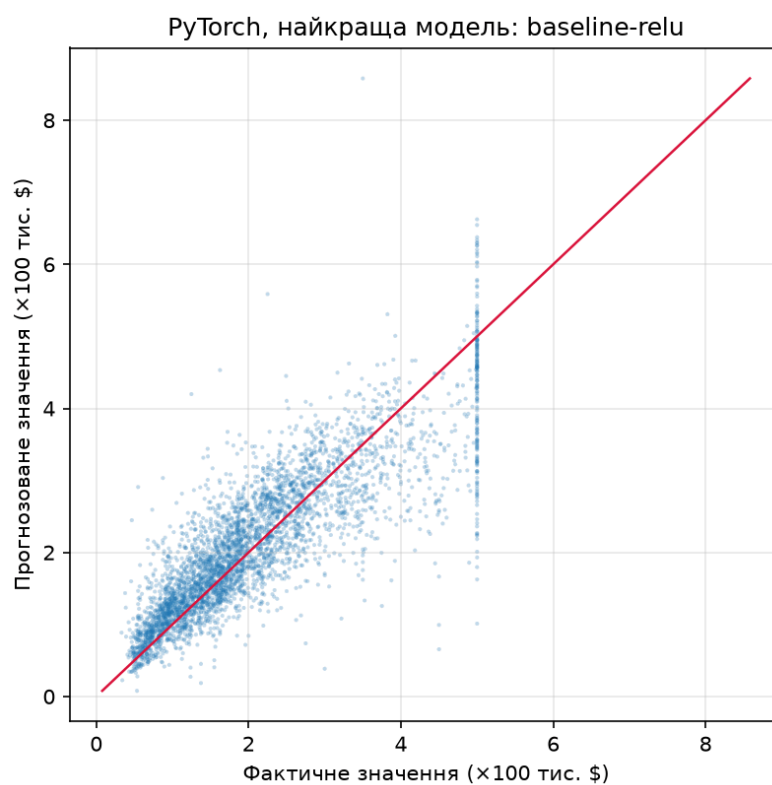


Рис. 3: *PyTorch: прогнозовані значення проти фактичних (baseline-relu)*

5. Порівняння TensorFlow та PyTorch

5.1. Якість моделей

Табл. 3: Порівняння RMSE на тестовій вибірці

Модель	TensorFlow	PyTorch	Різниця
baseline-relu	0,6082	0,6165	+1,4 %
baseline-gelu	0,6181	0,6228	+0,8 %
regularized-relu	0,7480	0,7542	+0,8 %
regularized-gelu	0,7770	0,7495	−3,5 %

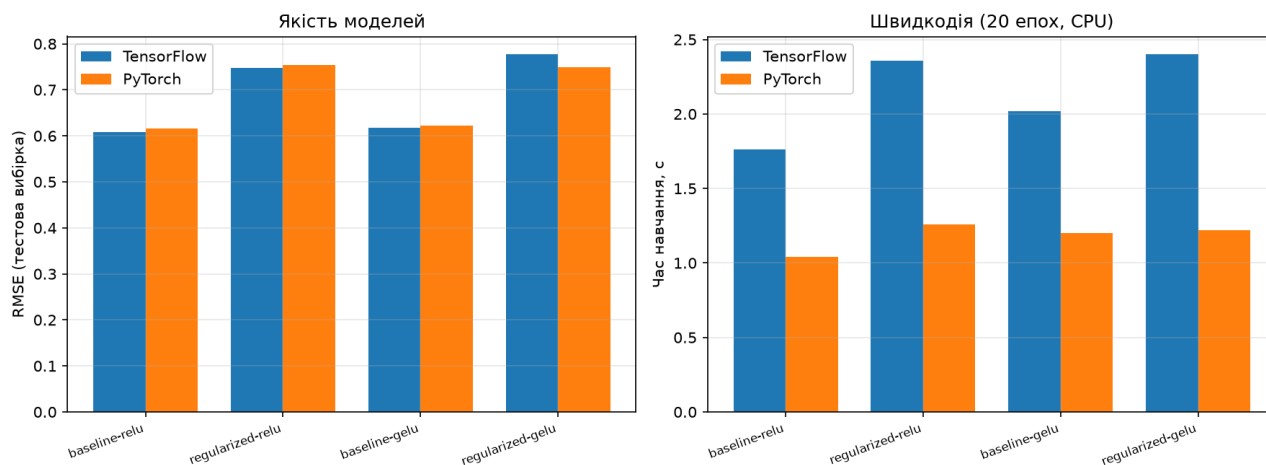


Рис. 4: Порівняння якості та швидкодії TensorFlow і PyTorch

Розбіжності між фреймворками не перевищують 3,5 % і не мають систематичного характеру: у трьох випадках дещо кращим виявився TensorFlow, в одному — PyTorch. Це узгоджується з очікуваннями: за ідентичних архітектури, гіперпараметрів і поділу даних обидва фреймворки реалізують один і той самий алгоритм, а залишкова різниця пояснюється різною ініціалізацією ваг та різним порядком формування батчів, які неможливо зробити побітово однаковими через різні генератори випадкових чисел.

5.2. Швидкодія

На CPU час навчання 20 епох склав 2,3–8,0 с для TensorFlow та 4,2–15,7 с для PyTorch. Виміри помітно зашумлені через конкуренцію за ядра процесора,

тому робити з них висновок про принципову перевагу котрогось фреймворку некоректно. Можна лише констатувати, що на задачі такого масштабу (12 тис. прикладів, 2,7 тис. параметрів) обидва фреймворки дають цілком прийнятний час навчання без використання GPU.

5.3. Відмінність у підрахунку параметрів

Окремо слід пояснити розбіжність у кількості параметрів регуляризованих моделей: 2 977 у TensorFlow проти 2 785 у PyTorch. Архітектури ідентичні, різниця складає рівно $2 \times (64 + 32) = 192$ значення.

Причина суто облікова. Шар BatchNorm має чотири набори величин: навчувані γ і β та ненавчувані ковзні середнє й дисперсію. Метод `model.count_params()` у Keras враховує всі чотири, тоді як `sum(p.numel() for p in model.parameters())` у PyTorch повертає лише навчувані, бо ковзні статистики зареєстровані як буфери, а не параметри. Отже, **моделі однакові**, різниться лише спосіб підрахунку.

5.4. Переваги та недоліки фреймворків

Табл. 4: Порівняння фреймворків за досвідом виконання роботи

Критерій	TensorFlow / Keras	PyTorch
Обсяг коду	Менший: <code>.fit()</code> інкапсулює весь цикл навчання	Більший: цикл навчання пишеться вручну
Контроль над навчанням	Нестандартна логіка потребує callback-ів або <code>train_step</code>	Повний контроль за за-мовчуванням
Регуляризація L2	На рівні шару (<code>kernel_regularizer</code>)	На рівні оптимізатора (<code>weight_decay</code>)
Режими train/eval	Перемикаються автоматично	Вимагають явних <code>.train()</code> / <code>.eval()</code>
Імовірні помилки	Приховування деталей ускладнює діагностику	Легко забути <code>zero_grad()</code> або <code>.eval()</code>
Налагодження	Складніше через рівень абстракції	Простіше: звичайний імперативний код Python

6. Аналіз архітектур та причин недонавчання

Обидва фреймворки дали узгоджений і спершу неочікуваний результат: **базова мережа переважає регуляризовану** приблизно на 22 % за RMSE.

Діагностика за рисунками 1 та 2 показує, що причиною є **недонавчання, а не перенавчання**. Розрив між валідаційною та тренувальною втратою наприкінці навчання не перевищує 0,081 і не зростає з часом, тобто перенавчання відсутнє. Натомість регуляризовані моделі демонструють високу втрату *одночасно* на тренувальній та валідаційній вибірках (0,472 і 0,553 проти 0,342 і 0,362 у базовій моделі) — класична ознака надмірного обмеження моделі.

Пояснення полягає у співвідношенні ємності моделі та обсягу даних. Мережа має близько 2,7 тис. параметрів на 12,4 тис. тренувальних прикладів, тобто даних

майже в'ятеро більше, ніж параметрів. За такого співвідношення перенавчання малоймовірне, і регуляризувати нічого. Dropout із коефіцієнтом 0,2 вимикає кожен п'ятий нейрон у шарах із 64 та 32 одиницями, що для мережі такого розміру є відчутною втратою ємності.

Висновок для практики: регуляризацію слід вводити як відповідь на діагностоване перенавчання, а не як обов'язковий елемент архітектури. Застосована «про всяк випадок», вона погіршує результат.

7. Висновки

У ході лабораторної роботи засобами PyTorch відтворено експеримент лабораторної роботи №1 та виконано порівняльний аналіз двох фреймворків.

1. Реалізовано дві архітектури (базову та регуляризовану) з двома функціями активації кожна. Найкращий результат показала базова мережа з ReLU: $RMSE = 0,617$, $R^2 = 0,711$.
2. Результати PyTorch і TensorFlow збігаються в межах 3,5 % і не виявляють систематичної переваги жодного з фреймворків. Залишкова розбіжність пояснюється різною ініціалізацією ваг та порядком формування батчів.
3. Виявлену розбіжність у кількості параметрів (2 977 проти 2 785) пояснено способом підрахунку: Keras враховує ковзні статистики BatchNorm, PyTorch — ні. Самі моделі ідентичні.
4. Основна практична відмінність фреймворків — рівень абстракції. Keras приховує цикл навчання за `.fit()`, що скорочує код, але ускладнює нестандартні сценарії. PyTorch вимагає написати цикл вручну, що збільшує обсяг коду та кількість потенційних помилок, але дає повний контроль і простіше налагодження.
5. В обох фреймворках регуляризація погіршила результат приблизно на 22 % за RMSE. Встановлено, що причина — недонавчання: обсяг даних переважає ємність моделі майже в'ятеро, тому перенавчання не виникає і регуляризація лише обмежує апроксимаційну здатність мережі.
6. Для цієї задачі вибір архітектури впливає на результат значно сильніше (різниця 22 %), ніж вибір функції активації (до 1,0 % у PyTorch та до 3,9 % у TensorFlow) чи вибір фреймворку (до 3,5 %).